

INCOEX INSTITUTO TÉCNICO
TÉCNICO EN CIENCIAS DE DATOS

MÓDULO 2: Manejo y Preparación de Datos

PROYECTO FINAL DEL MÓDULO 2
Manejo de Base de Datos con SQL Server

NOMBRE DEL ESTUDIANTE: Katherine Susana Rojas Saborío

Grupo 04

Fecha de entrega: 1 de agosto, 2026

Tabla de contenido

1. Base de datos	3
Script de la creación de la base de datos	3
2. Modelo Entidad-Relación (E-R)	3
3. Tablas	5
Script de la creación de las tablas	5
Justificación de la creación de las tablas y sus relaciones:	7
4. Script SQL	8
5. Registros.....	8
6. Consultas SQL	9
7. Vistas	15
8. Diagrama de SQL.....	18

1. Base de datos

Script de la creación de la base de datos

Se utilizó el motor SQL Server Management Studio 2022 para la realización de los scripts de SQL. Se presenta la creación de la base de datos de la siguiente manera:

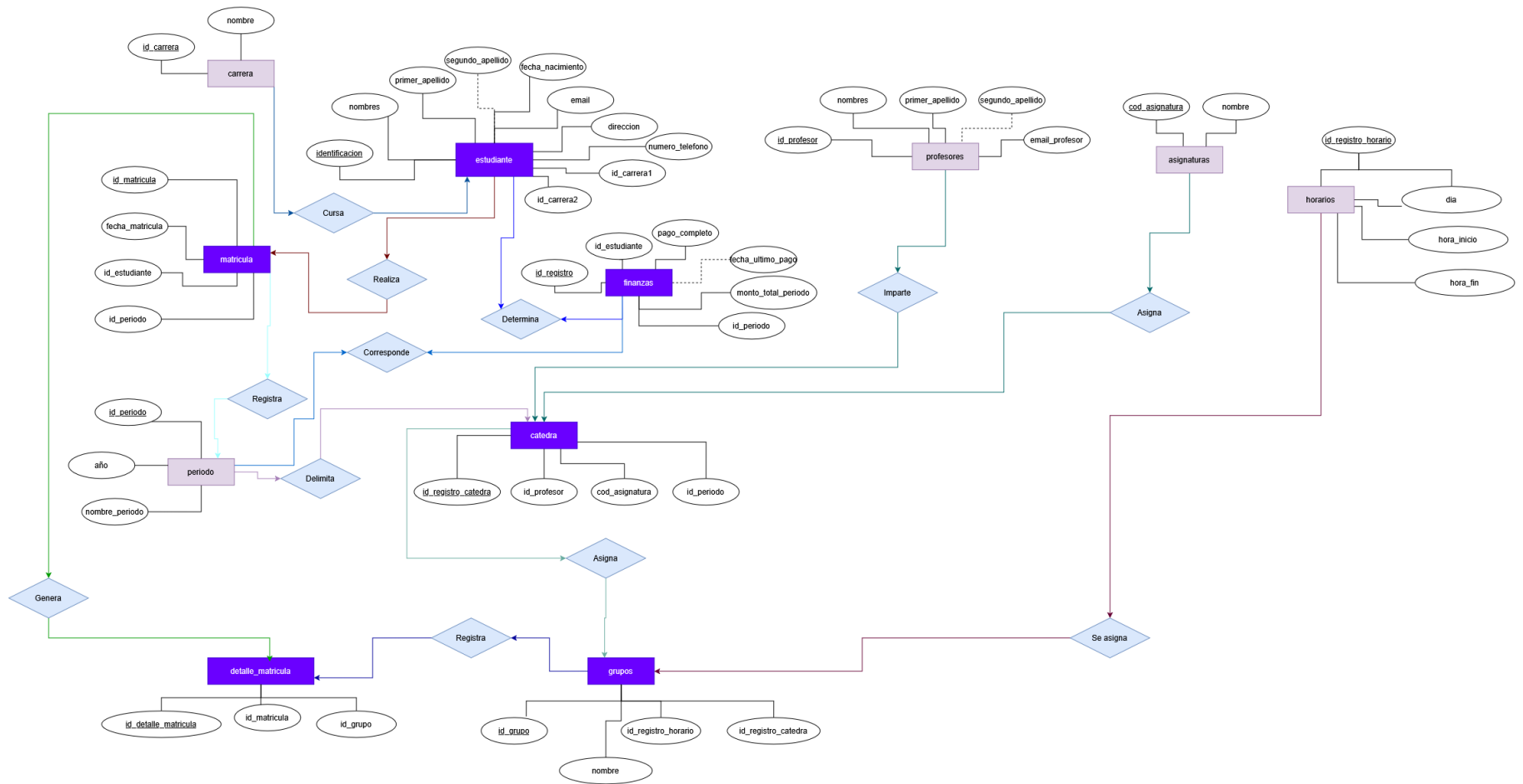
```
-- Creation of the database for the University, with the purpose of registering enrollment
CREATE DATABASE db_universidad;
GO

-- Select the database for further queries
USE db_universidad;
GO
```

2. Modelo Entidad-Relación (E-R)

El modelo fue realizado mediante la página draw.io, de forma anticipada a la creación de las tablas en SQL.

Muestra las entidades, atributos y relaciones entre tablas.



PROCESO DE MATRÍCULA DE UNA UNIVERSIDAD

3. Tablas

Script de la creación de las tablas

```
-- Creation of the tables according to the Entity Relationship Diagram
-- carrera will store the name and id of each career
CREATE TABLE carrera (
    id_carrera SMALLINT PRIMARY KEY,
    nombre VARCHAR(30) NOT NULL
)

-- estudiante stores each student's personal and contact info, identified by their
-- identification, which may have different formats if the student is international
CREATE TABLE estudiante (
    identificacion VARCHAR(20) PRIMARY KEY,
    nombres NVARCHAR(50) NOT NULL,
    primer_apellido NVARCHAR(15) NOT NULL,
    segundo_apellido NVARCHAR(15),
    fecha_nacimiento DATE NOT NULL,
    email VARCHAR(50) UNIQUE NOT NULL,
    direccion VARCHAR(200) NOT NULL,
    numero_telefono VARCHAR(20) UNIQUE NOT NULL,
    id_carrera1 SMALLINT NOT NULL,
    id_carrera2 SMALLINT,
    CONSTRAINT FK_carrera1 FOREIGN KEY (id_carrera1) REFERENCES carrera(id_carrera),
    CONSTRAINT FK_carrera2 FOREIGN KEY (id_carrera2) REFERENCES carrera(id_carrera)
)

-- Profesores stores each professor's basic information, identified by id_profesor, will be
-- used later to connect the catedra
CREATE TABLE profesores (
    id_profesor VARCHAR(20) PRIMARY KEY,
    nombres NVARCHAR(50) NOT NULL,
    primer_apellido NVARCHAR(15) NOT NULL,
    segundo_apellido NVARCHAR(15),
    email_profesor VARCHAR(50) NOT NULL
)

-- Stores the name and id of each subject offered in the university
CREATE TABLE asignaturas (
    cod_asignatura SMALLINT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL
)

-- Stores available time blocks (day, start and end time) that can be reused across different
-- groups
CREATE TABLE horarios (
    id_registro_horario SMALLINT IDENTITY(1,1) PRIMARY KEY,
    dia VARCHAR(15) NOT NULL,
    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL
)

-- Stores each academic period, which will be used later to tie enrollment, teaching and
-- payments to a specific term
CREATE TABLE periodo (
    id_periodo VARCHAR(10) PRIMARY KEY,
    año SMALLINT NOT NULL,
    nombre_periodo VARCHAR(25) NOT NULL
)

-- Registers the enrollments of the students according to the period, connected to estudiante
-- and periodo through FK
CREATE TABLE matricula (
    id_matricula SMALLINT IDENTITY(1,1) PRIMARY KEY,
    fecha_matricula DATE NOT NULL,
    id_estudiante VARCHAR(20) NOT NULL,
    id_periodo VARCHAR(10) NOT NULL,
```

```

        CONSTRAINT FK_estudiante_matricula FOREIGN KEY (id_estudiante) REFERENCES
estudiante(identificacion),
        CONSTRAINT FK_periodo_matricula FOREIGN KEY (id_periodo) REFERENCES
periodo(id_periodo)
)

-- Registers the payments of the students according to their total to pay during a period,
connected to estudiante and periodo through FK
CREATE TABLE finanzas (
    id_registro SMALLINT IDENTITY(1,1) PRIMARY KEY,
    id_estudiante VARCHAR(20) NOT NULL,
    id_periodo VARCHAR(10) NOT NULL,
    pago_completo BIT NOT NULL DEFAULT 0,
    fecha_ultimo_pago DATE,
    monto_total_periodo DECIMAL(10,2) NOT NULL,
    CONSTRAINT FK_estudiante_finanzas FOREIGN KEY (id_estudiante) REFERENCES
estudiante(identificacion),
    CONSTRAINT FK_periodo_finanzas FOREIGN KEY (id_periodo) REFERENCES
periodo(id_periodo)
)

-- Creation of intermediate tables to maintain normalization and coherent connection of each
register
-- Connects a professor to a subject within a specific period, defining who teaches what and
when, through FK to profesores, asignaturas and periodo
CREATE TABLE catedra (
    id_registro_catedra SMALLINT IDENTITY(1,1) PRIMARY KEY,
    id_profesor VARCHAR(20) NOT NULL,
    cod_asignatura SMALLINT NOT NULL,
    id_periodo VARCHAR(10) NOT NULL,
    CONSTRAINT FK_profesor_catedra FOREIGN KEY (id_profesor) REFERENCES
profesores(id_profesor),
    CONSTRAINT FK_asignatura_catedra FOREIGN KEY (cod_asignatura) REFERENCES
asignaturas(cod_asignatura),
    CONSTRAINT FK_periodo_catedra FOREIGN KEY (id_periodo) REFERENCES
periodo(id_periodo)
)

-- Represents a specific group of a catedra subject, assigning it a time block, connected to
catedra and horarios through FK
CREATE TABLE grupos (
    id_grupo SMALLINT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(15) NOT NULL,
    id_registro_horario SMALLINT NOT NULL,
    id_registro_catedra SMALLINT NOT NULL,
    CONSTRAINT FK_horario_grupo FOREIGN KEY (id_registro_horario) REFERENCES
horarios(id_registro_horario),
    CONSTRAINT FK_catedra_grupo FOREIGN KEY (id_registro_catedra) REFERENCES
catedra(id_registro_catedra) ON DELETE CASCADE
)

-- Connects a matricula to the specific groups the student enrolled in, since one enrollment
can include several groups, through FK to matricula and grupos
CREATE TABLE detalle_matricula (
    id_detalle_matricula SMALLINT IDENTITY(1,1) PRIMARY KEY,
    id_matricula SMALLINT NOT NULL,
    id_grupo SMALLINT NOT NULL,
    CONSTRAINT FK_matricula_detalle FOREIGN KEY (id_matricula) REFERENCES
matricula(id_matricula) ON DELETE CASCADE,
    CONSTRAINT FK_grupo_detalle FOREIGN KEY (id_grupo) REFERENCES grupos(id_grupo) ON
DELETE CASCADE
)

```

Justificación de la creación de las tablas y sus relaciones:

Para la base de datos de una universidad, enfocada en el proceso de matrícula, se crearon tablas en un orden lógico que respeta la integridad referencial y la normalización. En este proyecto se aplica la segunda forma normal (2FN) asegurando que todos los atributos no clave dependan totalmente de la clave primaria. Por otra parte, la primera (1FN) y tercera forma normal (3FN) se aplican parcialmente, ya que, aunque está presente en la mayoría de las tablas y garantiza la atomicidad de los datos, se realiza una excepción en la tabla estudiantes, la cual cuenta con la columna dirección, que guarda la dirección completa del estudiante.

Una vez aclarada la integridad de las tablas, se explica de forma breve la justificación de cada tabla:

carrera: esta tabla muestra el código y nombre de todas las carreras que se imparten en la universidad. No cuenta con dependencias.

estudiante: esta tabla almacena la información de todos los estudiantes que están inscritos en la universidad, su llave primaria es la identificación, la cual puede tener un formato variado al contemplar estudiantes nacionales o internacionales. Se conecta con carrera, puesto que un estudiante se empadrona en una carrera al inscribirse a la universidad. Un estudiante puede tener dos carreras. Será utilizada para conectarse con datos de finanzas y matrícula.

profesores: esta tabla almacena la información básica de los profesores, su llave primaria es la identificación, la cual, al igual que en estudiantes, puede tener un formato variado. Será utilizada para formar la cátedra de las asignaturas.

asignaturas: esta tabla almacena el nombre de las materias/asignaturas que se imparten en la universidad, identificadas por su código de asignatura.

horarios: esta tabla almacena los días y las horas en las que se pueden impartir clases en la universidad. Se identifica con un registro de fecha, que guarda un bloque de tiempo disponible.

periodo: esta tabla registra los periodos académicos que definen el ciclo lectivo de la universidad. Se identifican por el id del periodo, el cuál es único y representativo.

Las tablas anteriores, excepto carrera, no cuentan con llaves foráneas, pero definen los registros de las siguientes tablas:

matrícula: esta tabla almacena el evento de matrícula de un estudiante en la universidad en un periodo específico y se identifica con un id de matrícula. Depende de que el estudiante y el periodo existan dentro de las tablas periodo y universidad.

finanzas: cuando un estudiante realiza una matrícula, existe un proceso financiero asociado a la identificación del estudiante, quien debe pagar lo correspondiente para seguir cursando en los próximos periodos. Su llave primaria es el id del registro financiero que se crea cuando un estudiante establece su matrícula.

catedra: esta es una de las tablas más importantes y dependientes de la base de datos, ya que permite conectar dentro de un periodo específico las asignaturas disponibles y los profesores que imparten cada asignatura. Se identifica con un id del registro de cátedra, que se crea al asignar cuál profesor imparte determinadas materias en un periodo.

grupos: esta tabla permite separar los grupos de estudiantes que acudirán a las clases de acuerdo con los horarios disponibles y el registro de la cátedra (que contiene, como se mencionó anteriormente, profesores, asignatura y periodo). Se identifica con un id de grupo; sin embargo, se le asigna un nombre para identificarlo con mayor claridad desde una interfaz.

detalle_matrícula: esta tabla depende de la tabla grupo, que a su vez cuenta con muchas dependencias. Es necesaria porque un estudiante se matricula en varias asignaturas en cada periodo, y su elección estará asociada a la asignatura, al grupo que le corresponde, el horario y el profesor. Se identifica con un id de detalle de matrícula y almacena id de matrícula y grupo.

4. Script SQL

Dentro del repositorio, se encuentra un archivo llamado “Script.sql”, que contiene la creación de la base de datos y las tablas explicadas con anterioridad.

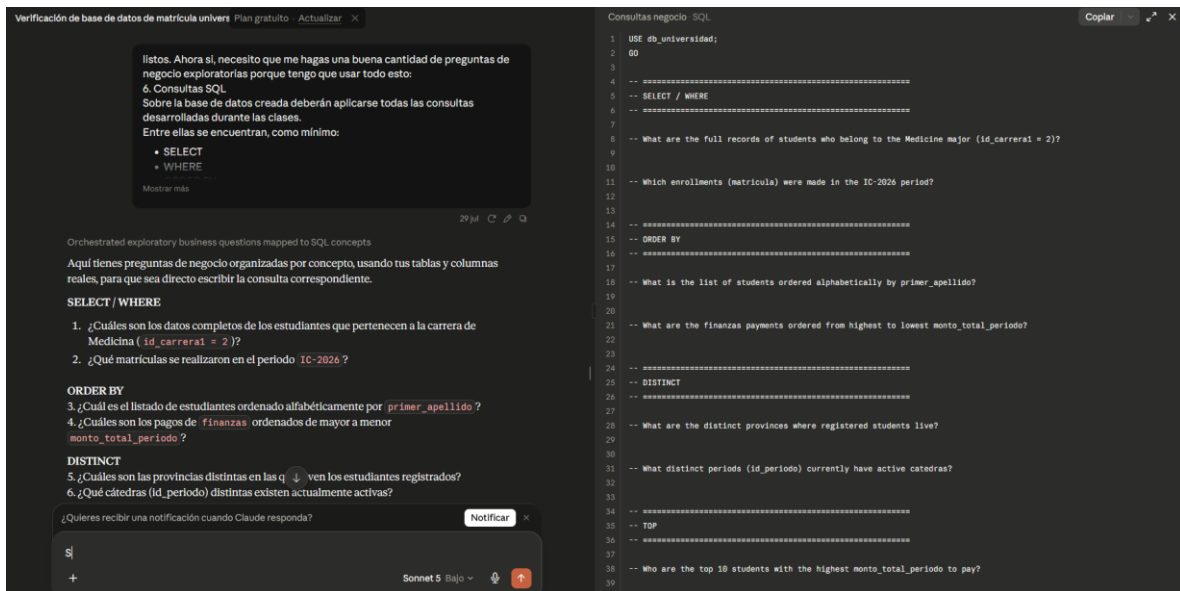
5. Registros

La inserción de los datos en las tablas se complementa con una inteligencia artificial (Claude IA), debido al gran volumen de datos necesarios para el funcionamiento de la base de datos. Se compartió el script de las tablas y se especificaron detalles hasta que la inserción fue coherente. El uso de inteligencia artificial debe ser cauteloso, puesto que existieron múltiples inconsistencias durante los primeros prompts, como una geografía incorrecta en la dirección de los estudiantes o la asignación manual de llaves primarias que ya contaban con IDENTITY(1,1).

El script de la inserción de datos se encuentra dentro del repositorio, con el nombre de “inserción_sql_proyecto.sql”.

6. Consultas SQL

Se ejecutaron múltiples consultas en SQL para obtener información de las tablas. Dada la lista de consultas solicitadas en el proyecto, se solicitó ayuda a “Claude IA” para generar una serie de preguntas de negocio relacionadas con los datos insertados. A continuación, se muestra la evidencia del origen de las preguntas de negocio:



Durante la realización de las consultas se hicieron múltiples modificaciones a preguntas predeterminadas por la IA para mantener la coherencia y el objetivo del proyecto, y se respondieron una a una de forma manual en SSMS. Se utilizó el conocimiento del curso complementado con el nivel avanzado de SQL reflejado en proyectos de portafolio de GitHub.

El script se encuentra en el archivo en el repositorio llamado “consultas_sql_proyecto.sql”.

```
-- =====
-- SELECT / WHERE
-- =====

-- What are the full records of students who belong to the Medicine major (id_carrera1 = 2)?
SELECT * FROM estudiante
WHERE id_carrera1 = 2 OR id_carrera2 = 2;
GO

-- Which enrollments (matricula) were made in the IC-2026 period?
SELECT * FROM matricula
WHERE id_periodo = 'IC-2026';

-- =====
-- ORDER BY
-- =====
```

```

-- What is the list of students ordered alphabetically by primer_apellido?
SELECT nombres, primer_apellido, segundo_apellido FROM estudiante
ORDER BY primer_apellido

-- What are the finanzas payments ordered from highest to lowest monto_total_periodo?
SELECT id_periodo, monto_total_periodo FROM finanzas
ORDER BY monto_total_periodo DESC;
GO

-- =====
-- DISTINCT
-- =====

-- What are the distinct primary majors associated with enrolled students?
SELECT DISTINCT
    c.nombre AS nombre_carrera
FROM estudiante e
INNER JOIN matricula m ON e.identificacion = m.id_estudiante
INNER JOIN carrera c ON c.id_carrera = e.id_carrera1;
GO

-- =====
-- TOP
-- =====

-- Who are the top 10 students with the highest monto_total_periodo to pay?
SELECT TOP 10
    e.identificacion,
    e.nombres,
    e.primer_apellido,
    e.segundo_apellido,
    f.monto_total_periodo
FROM estudiante e
INNER JOIN finanzas f ON e.identificacion = f.id_estudiante
ORDER BY monto_total_periodo DESC;
GO

-- =====
-- LIKE
-- =====

-- What subjects contain the word "Derecho" in their name?
SELECT * FROM asignaturas
WHERE nombre LIKE '%Derecho%';

-- What students have an email ending in "@estudiantes.ac.cr"?
SELECT
    identificacion,
    nombres,
    primer_apellido,
    segundo_apellido,
    email
FROM estudiante
WHERE email LIKE '%estudiantes.ac.cr';
GO

-- =====
-- BETWEEN
-- =====

-- What finanzas records have a monto_total_periodo between 300000 and 500000?
SELECT * FROM finanzas
WHERE monto_total_periodo BETWEEN 300000 AND 500000
ORDER BY monto_total_periodo

-- What classes start between 13:00 and 18:00?
SELECT
    c.id_periodo,

```

```

        c.cod_asignatura,
        a.nombre,
        g.nombre AS nombre_grupo,
        h.hora_inicio,
        h.hora_fin
FROM grupos g
INNER JOIN catedra c ON g.id_registro_catedra = c.id_registro_catedra
INNER JOIN asignaturas a ON a.cod_asignatura=c.cod_asignatura
INNER JOIN horarios h ON g.id_registro_horario = h.id_registro_horario
WHERE h.hora_inicio BETWEEN '13:00:00' AND '18:00:00';
GO

-- =====
-- IN
-- =====

-- What students belong to Computer Engineering, Civil Engineering or Graphic Design?
SELECT DISTINCT
    e.identificacion,
    e.nombres,
    e.primer_apellido,
    c.nombre AS nombre_carrera
FROM estudiante e
INNER JOIN carrera c
    ON c.id_carrera = e.id_carrera1
    OR c.id_carrera = e.id_carrera2
WHERE c.nombre IN ('Ingenieria en Computacion', 'Ingenieria Civil', 'Diseno Grafico');
GO

-- What subjects have the codes 101, 201 or 301?
SELECT * FROM asignaturas
WHERE cod_asignatura IN(101, 201, 301);
GO

-- =====
-- NOT
-- =====

-- What students do NOT belong to the Law major?
SELECT DISTINCT
    e.identificacion,
    e.nombres,
    e.primer_apellido,
    e.segundo_apellido,
    c.nombre
FROM
    estudiante e
INNER JOIN carrera c
    ON c.id_carrera = e.id_carrera1
    OR c.id_carrera = e.id_carrera2
WHERE NOT c.nombre = 'Derecho';
GO

-- What finanzas records do NOT have pago_completo = 1?
SELECT * FROM finanzas
WHERE NOT pago_completo = 1;
GO

-- =====
-- IS NULL / IS NOT NULL
-- =====

-- What students do NOT have a secondary major?
SELECT * FROM estudiante
WHERE id_carrera2 IS NULL;
GO

-- What students DO have a segundo_apellido registered?
SELECT * FROM estudiante

```

```

WHERE segundo_apellido IS NOT NULL;
GO

-- =====
-- AND / OR
-- =====

-- What students are from San Jose province AND belong to the Psychology major?
SELECT DISTINCT
    e.identificacion,
    e.nombres,
    e.primer_apellido,
    e.segundo_apellido,
    e.direccion,
    c.nombre AS nombre_carrera
FROM estudiante e
INNER JOIN carrera c
    ON c.id_carrera = e.id_carrera1
    OR c.id_carrera = e.id_carrera2
WHERE e.direccion LIKE '%San Jose%' AND c.nombre = 'Psicologia';
GO

-- What finanzas records have pago_completo = 0 OR fecha_ultimo_pago IS NULL?
SELECT * FROM finanzas
WHERE pago_completo = 0 OR fecha_ultimo_pago IS NULL;
GO

-- =====
-- GROUP BY
-- =====

-- How many students are there grouped by id_carrera1?
SELECT
    COUNT(*) AS total_estudiantes,
    id_carrera1
FROM estudiante
GROUP BY id_carrera1;
GO

-- How many subjects does each professor teach, grouped by id_profesor?
SELECT
    p.id_profesor,
    p.nombres,
    COUNT(a.nombre) AS asignaturas_imparturas
FROM profesores p
INNER JOIN catedra c ON p.id_profesor = c.id_profesor
INNER JOIN asignaturas a ON a.cod_asignatura = c.cod_asignatura
GROUP BY p.id_profesor, p.nombres;
GO

-- =====
-- HAVING
-- =====

-- What majors have more than 10 enrolled students?
SELECT
    c.nombre AS nombre_carrera,
    COUNT(e.id_carrera1) AS est_matriculados
FROM carrera c
INNER JOIN estudiante e ON e.id_carrera1 = c.id_carrera
INNER JOIN matricula m ON m.id_estudiante = e.identificacion
GROUP BY m.id_periodo, c.nombre
HAVING COUNT(e.id_carrera1) > 10;
GO

-- What professors teach more than 3 subjects?

```

```

SELECT
    p.nombres,
    p.primer_apellido,
    p.segundo_apellido,
    COUNT(c.cod_asignatura) AS cantidad_asignaturas_impartidas
FROM profesores p
INNER JOIN catedra c ON p.id_profesor=c.id_profesor
INNER JOIN asignaturas a ON a.cod_asignatura = c.cod_asignatura
GROUP BY p.nombres, p.primer_apellido, p.segundo_apellido
HAVING COUNT(c.cod_asignatura) > 3;
GO

-- =====
-- COUNT
-- =====
-- How many groups exist per subject?
SELECT
    a.nombre AS nombre_asignatura,
    COUNT(g.id_grupo) AS numero_grupos
FROM asignaturas a
INNER JOIN catedra c ON a.cod_asignatura = c.cod_asignatura
INNER JOIN grupos g ON g.id_registro_catedra = c.id_registro_catedra
GROUP BY a.nombre;
GO

-- =====
-- SUM
-- =====
-- What is the total amount to collect from all students in the IC-2026 period?
SELECT
    SUM(monto_total_periodo) AS total_recaudado_IC2026
FROM finanzas
WHERE id_periodo = 'IC-2026';
GO

-- What is the count of paid (pago_completo = 1) vs pending amounts per major?
SELECT DISTINCT
    c.nombre AS nombre_carrera,
    SUM(CASE WHEN f.pago_completo = 1 THEN 1 END) AS pagos_completos,
    SUM(CASE WHEN f.pago_completo = 0 THEN 1 END) AS pagos_pendientes
FROM finanzas f
INNER JOIN estudiante e ON e.identificacion = f.id_estudiante
INNER JOIN carrera c
    ON c.id_carrera = e.id_carrera1
    OR c.id_carrera = e.id_carrera2
GROUP BY c.nombre;
GO

-- =====
-- AVG
-- =====
-- What is the average monto_total_periodo students pay per period?
SELECT
    id_periodo,
    AVG(monto_total_periodo) AS promedio_por_periodo
FROM finanzas
GROUP BY id_periodo;
GO

-- =====
-- MIN / MAX
-- =====
-- What is the minimum and maximum amount paid in finanzas?
SELECT DISTINCT

```

```

        MIN(monto_total_periodo) AS minimo_pago,
        MAX(monto_total_periodo) AS maximo_pago
FROM finanzas;
GO

-- What is the oldest and most recent payment registered?
SELECT DISTINCT
    MIN(fecha_ultimo_pago) AS pago_mas_antiguo,
    MAX(fecha_ultimo_pago) AS pago_mas_reciente
FROM finanzas

-- =====
-- INNER JOIN
-- =====

-- Which student is enrolled in which group, showing student name and group name?
SELECT DISTINCT
    e.identificacion,
    e.nombres,
    e.primer_apellido,
    e.segundo_apellido,
    g.nombre AS nombre_grupo
FROM estudiante e
INNER JOIN matricula m ON e.identificacion = m.id_estudiante
INNER JOIN detalle_matricula d ON m.id_matricula = d.id_matricula
INNER JOIN grupos g ON g.id_grupo = d.id_grupo

-- Which professor teaches which subject, showing professor name and subject name?
SELECT DISTINCT
    p.id_profesor,
    p.nombres,
    p.primer_apellido,
    p.segundo_apellido,
    a.nombre AS nombre_asignatura
FROM profesores p
INNER JOIN catedra c ON p.id_profesor = c.id_profesor
INNER JOIN asignaturas a ON a.cod_asignatura = c.cod_asignatura

-- =====
-- LEFT JOIN
-- =====

-- What are all students and their finanzas records, including those who don't have a payment
record yet?
SELECT
    e.identificacion,
    e.nombres,
    e.primer_apellido,
    e.segundo_apellido,
    e.email,
    e.fecha_nacimiento,
    e.direccion,
    e.id_carrera1,
    e.id_carrera2,
    f.fecha_ultimo_pago,
    f.pago_completo,
    f.monto_total_periodo
FROM estudiante e
LEFT JOIN finanzas f ON e.identificacion=f.id_estudiante

-- What are all subjects and their groups, including subjects that don't have a group assigned
yet?
SELECT DISTINCT
    a.nombre,
    g.nombre AS nombre_grupo
FROM asignaturas a
LEFT JOIN catedra c ON a.cod_asignatura = c.cod_asignatura

```

```

LEFT JOIN grupos g ON g.id_registro_catedra = c.id_registro_catedra;
GO

-- =====
-- RIGHT JOIN
-- =====

-- What are all groups and their schedule, even if a schedule had no group associated?
SELECT DISTINCT
    g.nombre AS nombre_grupo,
    h.id_registro_horario,
    h.dia,
    h.hora_inicio,
    h.hora_fin
FROM grupos g
RIGHT JOIN horarios h ON g.id_registro_horario = h.id_registro_horario;
GO

-- What are all majors and the students they have, even if a major had no students?
SELECT DISTINCT
    c.id_carrera,
    c.nombre AS nombre_carrera,
    e.identificacion AS id_estudiante,
    e.nombres AS nombre_estudiante,
    e.primer_apellido
FROM estudiante e
RIGHT JOIN carrera c
    ON e.id_carrera1 = c.id_carrera
    OR e.id_carrera2 = c.id_carrera

-- =====
-- SUBQUERIES
-- =====

-- What are the provinces of the students?
WITH estudiante_provincia AS (
    SELECT
        LEFT(direccion, CHARINDEX(',', direccion)-1) AS provincia
    FROM estudiante
)
SELECT
    provincia,
    COUNT(*) AS total_estudiantes
FROM estudiante_provincia
GROUP BY provincia
ORDER BY provincia;
GO

-- What students have a monto_total_periodo higher than the general average?
SELECT
    e.identificacion,
    e.nombres,
    e.primer_apellido,
    e.segundo_apellido,
    f.monto_total_periodo
FROM estudiante e
INNER JOIN finanzas f ON e.identificacion = f.id_estudiante
WHERE f.monto_total_periodo > (SELECT AVG(monto_total_periodo) FROM finanzas);
GO

```

7. Vistas

El script “consultas_sql_proyecto.sql” finaliza con 4 vistas que permiten revisar información de la universidad útil para el personal administrativo, como el estado de

pago de los estudiantes en texto (“pago pendiente” o “pago completo”) en lugar de 0 y 1 que proviene del dato de tipo BIT, así como el nombre completo. También se crea una vista que muestra el horario de todas las clases que se imparten en el IC-2026 con su respectivo grupo, profesor y asignatura. Otra vista resume el número de estudiantes matriculados según su carrera. La última realiza una categorización breve de los cursos por carrera, los cuales no están conectados mediante ninguna tabla, ya que lo usual es que una asignatura sea cursada por múltiples carreras y se prefiere mantener la simplicidad por este proyecto.

```
-- A view showing student id, full name, majors and payment status
CREATE VIEW estudiantes_pago
AS
    SELECT
        e.identificacion,
        CONCAT(e.nombres, ' ', e.primer_apellido, ' ', ISNULL(e.segundo_apellido, '')) AS
nombre_completo,
        c1.nombre AS carrera_primaria,
        ISNULL(c2.nombre, 'Sin carrera secundaria') AS carrera_secundaria,
        CASE f.pago_completo
            WHEN 0 THEN 'Pago pendiente'
            WHEN 1 THEN 'Pago completo'
        END AS estado_pago,
        f.monto_total_periodo
    FROM estudiante e
    FULL OUTER JOIN finanzas f ON e.identificacion = f.id_estudiante
    LEFT JOIN carrera c1 ON c1.id_carrera = e.id_carrera1
    LEFT JOIN carrera c2 ON c2.id_carrera = e.id_carrera2;
GO

SELECT * FROM estudiantes_pago
WHERE estado_pago = 'Pago pendiente';
GO

-- A view of the schedule of all the classes across the IC-2026
CREATE VIEW horario_clases
AS
    SELECT DISTINCT
        c.cod_asignatura,
        a.nombre AS nombre_asignatura,
        g.nombre AS nombre_grupo,
        CONCAT(p.nombres, ' ', p.primer_apellido, ' ', ISNULL(p.segundo_apellido, '')) AS
nombre_profesor,
        h.dia,
        h.hora_inicio,
        h.hora_fin
    FROM asignaturas a
    LEFT JOIN catedra c
        ON a.cod_asignatura = c.cod_asignatura
        AND c.id_periodo = 'IC-2026'
    LEFT JOIN profesores p ON p.id_profesor = c.id_profesor
    FULL OUTER JOIN grupos g ON g.id_registro_catedra = c.id_registro_catedra
    LEFT JOIN horarios h ON h.id_registro_horario = g.id_registro_horario
GO

SELECT * FROM horario_clases
WHERE dia = 'Viernes';
GO
```

```

-- A view summarizing total student enrollment per group and subject across the IC-2026
CREATE VIEW cant_estudiantes_cursos
AS
SELECT
    a.cod_asignatura,
    a.nombre AS nombre_asignatura,
    COUNT(d.id_detalle_matricula) AS estudiantes_matriculados
FROM asignaturas a
INNER JOIN catedra c ON a.cod_asignatura = c.cod_asignatura
INNER JOIN grupos g ON g.id_registro_catedra = c.id_registro_catedra
LEFT JOIN detalle_matricula d ON d.id_grupo = g.id_grupo
WHERE id_periodo = 'IC-2026'
GROUP BY
    a.cod_asignatura,
    a.nombre;
GO

SELECT * FROM cant_estudiantes_cursos
ORDER BY estudiantes_matriculados DESC;
GO

-- A view showing categories of subjects according to the career
SELECT * FROM asignaturas;
GO

SELECT * FROM carrera;
GO

CREATE VIEW asignatura_por_carrera
AS
SELECT
    cod_asignatura,
    nombre,
    CASE
        WHEN cod_asignatura BETWEEN 100 AND 199 THEN 'Derecho'
        WHEN cod_asignatura BETWEEN 200 AND 299 THEN 'Medicina'
        WHEN cod_asignatura BETWEEN 300 AND 399 THEN 'Ingenieria en Computacion'
        WHEN cod_asignatura BETWEEN 400 AND 499 THEN 'Administracion de Empresas'
        WHEN cod_asignatura BETWEEN 500 AND 599 THEN 'Arquitectura'
        WHEN cod_asignatura BETWEEN 600 AND 699 THEN 'Psicologia'
        WHEN cod_asignatura BETWEEN 700 AND 799 THEN 'Enfermeria'
        WHEN cod_asignatura BETWEEN 800 AND 899 THEN 'Contaduria Publica'
        WHEN cod_asignatura BETWEEN 900 AND 999 THEN 'Ingenieria Civil'
        WHEN cod_asignatura BETWEEN 1000 AND 1099 THEN 'Diseno Grafico'
    END AS carrera_correspondiente
FROM asignaturas;
GO

SELECT * FROM asignatura_por_carrera
WHERE carrera_correspondiente = 'Enfermeria';
GO

```

8. Diagrama de SQL

